Module 8 Lab Session 3: Reactive Forms and Live API

Module M8 Angular 22 Fundamentals
Session 3 of 3

Excercise 5 The Enrollment Form

Liya wants to enroll. She needs a form that captures her Student ID, the term, and optional backup course choices. The form must validate inputs before they reach the .NET API.

Step 1: Generate the Form Component

```
ng generate component features/enrollment-form
```

Step 2: Build the Form Model

This step has the most new concepts in the entire workbook. Read the inline comments carefully:

In `enrollment-form.component.ts`:

```
import { Component, inject, signal } from "@angular/core";
import {
  FormBuilder,
  FormControl,
  Validators,
  ReactiveFormsModule,
  FormArray,
} from "@angular/forms";

@Component({
  selector: "app-enrollment-form",
  standalone: true,
  imports: [ReactiveFormsModule], // Required  without this, Angular does not
recognize form directives
  templateUrl: "./enrollment-form.component.html",
})
export class EnrollmentFormComponent {
  // inject(FormBuilder) is Angular's way of requesting a service.
  // It is similar to constructor injection in .NET (like inject ILogger in a
C# class).
  // The "private" keyword means only this class can access it.
  private fb = inject(FormBuilder);
```

```
  // A signal to track whether the form was submitted (for showing a success
message)
  submitted = signal(false);

  // fb.nonNullable.group({...}) creates a form object in TypeScript code.
  // "nonNullable" ensures that all values are typed as 'string' instead of
'string | null'
  //  this saves you from writing null-checking code everywhere.
  //
  // Each field is defined as: [defaultValue, validators]
  // Validators are rules that the value must pass before the form is
considered valid.
  form = this.fb.nonNullable.group({
    studentId: [
      "",
      [Validators.required, Validators.pattern("^STU-[0-9]{4}$")],
    ],
    //          ^^ default value is empty string
    //               ^^ two validators: the field is required AND must match
the pattern STU-1234
    courseId: ["", Validators.required],
    term: ["Fall 2026", Validators.required], // Pre-filled with a default
term
    notes: [""], // No validators  this field is optional
    backupCourses: this.fb.array<FormControl<string>>([]), // Starts empty,
user adds rows dynamically
  });

  // "get backups()" is a TypeScript property accessor  it looks like a
variable but runs a function.
  // This is a shortcut so you can write "this.backups" instead of
"this.form.controls.backupCourses"
  get backups() {
    return this.form.controls.backupCourses;
  }

  // Adds a new empty text input to the backup courses array
  addBackup() {
    this.backups.push(
      this.fb.control("", {
        nonNullable: true,
        validators: Validators.required,
      }),
    );
  }

  // Removes a specific backup course row by its position in the array
  removeBackup(index: number) {
    this.backups.removeAt(index);
```

```
  }

  submit() {
    if (this.form.valid) {
      // getRawValue() extracts the full form data as a JSON object.
      // IMPORTANT: Do NOT use .value here. If any field is disabled, .value
silently
      // drops that field from the object. getRawValue() always includes
everything.
      const payload = this.form.getRawValue();
      console.log("Enrollment payload:", payload);
      this.submitted.set(true);
    } else {
      // markAllAsTouched() forces Angular to show validation errors on every
field.
      // Without this call, Angular only shows errors on fields the user has
clicked on.
      this.form.markAllAsTouched();
    }
  }
}
```

Step 3: Build the Form Template

In `enrollment-form.component.html`:

```
<h2>Course Enrollment</h2>

@if (submitted()) {
<div class="success">
  Enrollment submitted. Check the console for the payload.
</div>
} @else {
<!-- [formGroup]="form" connects this <form> tag to the TypeScript form
object you built above. -->
<!-- (ngSubmit)="submit()" calls your submit() method when the user presses
Enter or clicks the submit button. -->
<form [formGroup]="form" (ngSubmit)="submit()">
  <label for="studentId">Student ID</label>
  <!-- formControlName="studentId" links this input to the 'studentId' field
in the form group. -->
  <!-- Angular keeps the TypeScript value and the on-screen input in sync
automatically. -->
  <input
    id="studentId"
    formControlName="studentId"
    placeholder="e.g. STU-1234"
  />
  <!-- Show the error ONLY when the user has clicked into and out of the
field (.touched) -->
```

```
  <!-- AND the current value fails validation (.invalid). -->
  @if (form.controls.studentId.touched && form.controls.studentId.invalid) {
  <span class="error">Enter a valid Student ID (format: STU-0000)</span>
  }

  <label for="courseId">Course ID</label>
  <input
    id="courseId"
    formControlName="courseId"
    placeholder="e.g. 1 (TMS course primary key)"
  />
  @if (form.controls.courseId.touched && form.controls.courseId.invalid) {
  <span class="error">Course ID is required</span>
  }

  <label for="term">Term</label>
  <input id="term" formControlName="term" />

  <label for="notes">Notes (optional)</label>
  <textarea id="notes" formControlName="notes"></textarea>

  <h3>Backup Courses</h3>
  <!-- $index is a built-in variable inside @for loops  the current position
(0, 1, 2...) -->
  @for (backup of backups.controls; track $index) {
  <div class="backup-row">
    <!-- [formControl]="backup" connects this input to the specific
FormControl object. -->
    <!-- This is different from formControlName  formControlName looks up by
string name, -->
    <!-- while [formControl] binds directly to the control object in the
array. -->
    <input
      [formControl]="backup"
      [placeholder]="'Backup course ' + ($index + 1)"
    />
    <!-- type="button" prevents this from submitting the form. -->
    <!-- Without type="button", any <button> inside a <form> defaults to
type="submit". -->
    <button type="button" (click)="removeBackup($index)">Remove</button>
  </div>
  }
  <button type="button" (click)="addBackup()">Add Backup Course</button>

  <hr />
  <!-- [disabled]="form.invalid" disables the button when ANY field fails
validation. -->
  <button type="submit" [disabled]="form.invalid">Confirm Enrollment</button>
```

```
</form>
}
```

Step 4: Route to the Form

Add a route in `app.routes.ts`:

```
{
  path: 'enroll',
  loadComponent: () =>
import('./features/enrollment-form/enrollment-form.component')
    .then(m => m.EnrollmentFormComponent)
}
```

Open the form at **`http://localhost:4200/enroll`**, or add a **`routerLink`** on the dashboard (for example an “Enroll” link to `/enroll`) so the page is easy to find the workbook adds the route but does not add that link for you.

Things That Will Go Wrong (and How to Fix Them)

- **Validation messages do not appear:** You are probably checking `.invalid` without checking `.touched`. Angular does not flag pristine (never-clicked) fields as errors. Call `.markAllAsTouched()` on submit.
- **`formGroup` directive not recognized:** You forgot to add `ReactiveFormsModule` to the component’s `imports` array.
- **Trying to use `[(ngModel)]` alongside `[formGroup]`:** This throws an error. Pick one form strategy per form element. For this form, use only Reactive Forms directives (`formControlName`, `[formControl]`).

Checkpoint 5

- ☐ The form renders with Student ID, Course ID, Term, and Notes fields
- ☐ Clicking “Add Backup Course” adds a new input row
- ☐ Clicking “Remove” removes that specific row
- ☐ Submitting with an empty Student ID shows the validation error
- ☐ A valid submission logs the complete payload to the console (open DevTools to verify)

Excercise 6 Connecting to the .NET API

Mock data got you this far. Now you connect to the real backend.

Before HTTP: Observable → async result (read this once)

So far, **`signal`**, **`computed`**, **`input`**, and **`output`** behaved like **synchronous** values in the component: the template reads them, Angular re-renders when they change. **`HttpClient.get()`** does **not** return a `Course[]` immediately. It returns an **Observable** a lazy stream that may emit **one value later**, then complete, or emit an **error**. Someone has to **subscribe** to start the request and receive the payload. If you subscribe by hand

in a component and forget to tear down, you get the memory-leak scenario from the **M8 Assessment Pack** scenarios.

`rxResource` (from `@angular/core/rxjs-interop`) is Angular's bridge: you keep the **Observable** inside `CourseService`, but the component exposes **`coursesResource.value()`**, **`.isLoading()`**, and **`.error()`** as **signals** the template already knows how to use. Same mental model as the rest of the workbook only the **data source** is asynchronous.

Mental model: `Observable` = "a future stream of results." `rxResource` = "run that stream when the resource loads, surface outcomes as signals, and clean up when this component goes away." You do **not** need to master all of RxJS for this lab; you need this **one** pattern.

Step 1: Verify Your API Is Running

Open a separate terminal and start the **same TMS Web API** you built in Module 6 (and extended in Module 7 if your cohort ran those topics):

```
cd path/to/your/tms-api
dotnet run
```

Confirm you get **200** with a JSON body. Send **`GET`** with **`page`** and **`pageSize`** (for example `https://localhost:5001/api/courses?page=1&pageSize=20`); adjust host/port if your API prints something else.

Catalogue envelopes (Angular mapping) read once, verify once in Scalar:

- **Never integrate** against a bare root JSON array **`[ {...}, {...} ]`** for TMS: the catalogue is always wrapped in an object.
- **`GET /api/courses`** (M6 spine you still run in many cohorts) and **`GET /api/v1/courses`** (once URL versioning is live): **`items[]`** carries the rows. Paging totals usually live **alongside** **`items`** on that same envelope (**`totalCount`**, **`page`**, **`pageSize`**, **`totalPages`**, **`hasPrevious`**, **`hasNext`**, camelCase JSON).
- **`GET /api/v2/courses`** (the programme M7 hardened catalogue envelope): **`data[]`** carries the rows; paging often nests under **`meta`**, hypermedia paths under **`links`**. Inspect your real payload, your service must map whatever field actually holds **`Course[]`**.

Every list row exposes **`id`**, **`code`**, **`title`**, **`maxCapacity`**, **`enrollmentCount`**. This Excercise 6 scaffold calls **`GET /api/courses`** and maps **`p.items`**; if your integration URL is **`/api/v2/courses`**, change the map to **`p.data`** (and extend your TypeScript type to match **`meta`** / **`links`** if TypeScript starts complaining).

Use Scalar, Postman, `curl`, or `Invoke-RestMethod`. If it does not respond, fix the API before continuing, this Excercise is about the Angular side, not debugging .NET.

Step 2: Create the Course Service

```
ng generate service services/course
```

This creates `src/app/services/course.service.ts`. The CLI generates a minimal skeleton. Replace it with:

```
import { Service, inject } from "@angular/core";
import { HttpClient } from "@angular/common/http";
import { map } from "rxjs/operators";
import { Course, CourseDetail, PagedResponse } from "../models/course.model";

// @Service() means Angular creates one instance of this service
// and shares it across the entire app. This is the Angular 22 shorthand
replacing legacy @Injectable.
// This is similar to AddSingleton<T>() in .NET's dependency injection.
@Service()
export class CourseService {
  // inject(HttpClient) requests Angular's HTTP client  the same pattern as
inject(FormBuilder)
  private http = inject(HttpClient);
  private baseUrl = "https://localhost:5001/api/courses";

  getAll() {
    // This URL is GET /api/courses → map items[] (M6 catalogue envelope).
Never accept a bare root [...].
    // Switch to map((p) => p.data) if your base URL is GET
/api/v2/courses—paging often nests under meta on that envelope (Step 1).
    return this.http
      .get<PagedResponse<Course>>(this.baseUrl, {
        params: { page: "1", pageSize: "50" },
      })
      .pipe(map((p) => p.items));
  }

  getById(id: string) {
    return this.http.get<CourseDetail>(`${this.baseUrl}/${id}`);
  }
}
```

Step 3: Consume the Service with rxResource

Open your **existing** `student-dashboard.component.ts` from Labs 1–3 do **not** start a second component file. **Merge** the changes below into the class you already have: add **`inject`** to your `@angular/core` import if it is not there yet (alongside `Component`, `signal`, `computed`), add the imports below, remove the **`availableCourses`** signal (and any code that exists only to feed the old mock catalog), add **`coursesResource`**, and keep **`handleEnroll`**, **`selectedCourse`**, and your `@Component` `imports` (for example

`CourseCardComponent`) intact. If you paste a second `export class StudentDashboardComponent`, TypeScript will error edit the one class.

At the top, add these imports (merge with existing lines, do not duplicate):

```
import { rxResource } from "@angular/core/rxjs-interop";
import { CourseService } from "../../services/course.service";
```

Now replace the hardcoded `availableCourses` signal with a live API call. The key change is using `rxResource` this is Angular 22's recommended way to safely fetch data from an API, where it is fully stable:

```
export class StudentDashboardComponent {
  // inject(CourseService) requests the service we just created.
  // Angular finds the singleton instance and gives it to us.
  private api = inject(CourseService);

  studentName = signal("Liya Kebede");
  earnedCredits = signal(45);

  graduationStatus = computed(() =>
    this.earnedCredits() >= 120 ? "Eligible for Graduation" : "In Progress",
  );

  // rxResource wraps the HTTP call into three managed signals:
  //   - coursesResource.isLoading() → true while waiting for the server
response
  //   - coursesResource.error()     → the error object if the request fails
  //   - coursesResource.value()     → the Course[] array when the request
succeeds
  //
  // It handles subscribing (starting the request) and unsubscribing
(cleaning up
  // if the user navigates away before the response arrives) automatically.
  // You never write .subscribe() or .unsubscribe() with rxResource.
  coursesResource = rxResource({
    loader: () => this.api.getAll(),
  });

  // ...rest of the component (handleEnroll, etc.)
}
```

Step 4: Update the Template

Replace the `availableCourses()` references with `coursesResource`:

```
<h2>Course Catalog</h2>

@if (coursesResource.isLoading()) {
<div class="spinner">Fetching courses from the server...</div>
```

```
} @else if (coursesResource.error()) {
<div class="error">
  Could not load courses. Make sure your .NET API is running.
</div>
} @else {
<div class="grid">
  @for (course of coursesResource.value()!; track course.id) {
  <tms-course-card [course]="course" (enrollClicked)="handleEnroll($event)"
/>
  } @empty {
  <p>No courses are available this term.</p>
  }
</div>
}
```

One new thing here: the `!` after `coursesResource.value()!`. This is TypeScript's **non-null assertion operator**. It tells TypeScript: "I know this value is not null at this point." We can safely use it here because the `@else` block only runs after `isLoading()` and `error()` are both false meaning the data has loaded successfully and `value()` contains the actual array.

Step 5: Handle CORS (You Will Hit This)

When the Angular app on `localhost:4200` tries to call the .NET API on `localhost:5001`, the browser will block the request with a CORS error. This is the browser's security policy it has nothing to do with Angular.

The fix is on the .NET side. In your TMS API `Program.cs`, ensure you have a CORS policy that allows `localhost:4200`:

```
builder.Services.AddCors(options =>
{
    options.AddPolicy("AllowAngular", policy =>
        policy.WithOrigins("http://localhost:4200")
              .AllowAnyHeader()
              .AllowAnyMethod());
});

// ...

app.UseCors("AllowAngular");
```

Step 6: Verify in the Browser

Open `http://localhost:4200/dashboard`. Open DevTools (F12) and go to the Network tab.

You should see a `GET` request to the URL in your `CourseService` (for example `https://localhost:5001/api/courses?page=1&pageSize=50`). The response should be

`200 OK` with a **wrapped catalogue** (not a root-level array): **`items[]`** carries rows on **`/api/courses`** and **`/api/v1/courses`**, while **`GET /api/v2/courses`** on the programme M7 envelope uses **`data[]`** (often with **`meta`** / **`links`**). Rows expose **`id`**, **`code`**, **`title`**, **`maxCapacity`**, **`enrollmentCount`** in camelCase. After mapping in the UI, course cards render with live rows—not the earlier mock array.

Checkpoint 6

- ☐ The Network tab shows a successful `GET` request to the .NET API
- ☐ Course cards render with real data (not the hardcoded mock array)
- ☐ The loading spinner appears briefly before data renders
- ☐ If you stop the .NET API and refresh, the error message appears instead